

WEEK-3

EXPERIMENT NO: 3

1. TITLE OF THE EXPERIMENT:

Implementation of a Student Name Manager using HTML, CSS and JavaScript

2. MAPPED OUTCOMES:

- Course Outcome (CO): CO1

3. AIM:

To design and implement a Student Name Manager using HTML, CSS, and JavaScript that allows users to add student names, display the list of students, count the total number of students, and delete students dynamically.

4. PROCEDURE:

Step 1: Create the HTML structure with a heading, text input, Add Student button, student count, and list.

Step 2: Create a JavaScript array to store student names.

Step 3: Implement the addStudent() function to read the name entered by the user and add it to the array.

Step 4: Validate the input and display an alert if the student name is empty.

Step 5: Implement the displayStudents() function to dynamically display all student names in the list.

Step 6: Display the total number of students using the array length.

Step 7: Implement the deleteStudent() function to remove a selected student from the array.

Step 8: Create a CSS file to style the webpage, input fields, buttons, student list, and delete buttons.

Step 9: Open index.html in a browser and verify the Add, Delete, and Count operations.

5. CODE:

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Name Manager</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
  <h1>Student Name Manager</h1>
```

```

<input type="text" id="studentName"
  placeholder="Enter student name">
<button onclick="addStudent()">Add Student</button>
<h3>Total Students:
  <span id="count">0</span>
</h3>
<ul id="studentList"></ul>
</div>
<script src="script.js"></script>
</body>
</html>

```

script.js

```

let students = [];

function addStudent() {
  const input = document.getElementById("studentName");
  const name = input.value.trim();

  if (name === "") {
    alert("Enter student name");
    return;
  }

  students.push(name);
  input.value = "";
  displayStudents();
}

function displayStudents() {
  const list = document.getElementById("studentList");
  list.innerHTML = "";

  students.forEach((student, index) => {
    list.innerHTML += `
      <li>
        ${index + 1}. ${student}
        <button class="delete"
          onclick="deleteStudent(${index})">
          Delete
        </button>
      </li>
    `;
  });

  document.getElementById("count").textContent = students.length;
}

function deleteStudent(index) {
  students.splice(index, 1);
  displayStudents();
}

```

style.css

```
body {
  font-family: Arial, sans-serif;
  background: #f4f4f4;
  margin: 0;
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}

.container {
  width: 400px;
  text-align: center;
  background: white;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 4px 10px #0003;
}

input {
  padding: 8px;
  width: 220px;
  border: 1px solid #ccc;
  border-radius: 5px;
}

button {
  padding: 8px 12px;
  cursor: pointer;
  margin-left: 5px;
  border: none;
  border-radius: 5px;
  background: #257488;
  color: white;
}

ul {
  list-style: none;
  padding: 10px;
  margin-top: 20px;
}

li {
  margin: 10px 0;
  background: #8ecae6;
  border-radius: 5px;
  padding: 10px;
  display: flex;
  justify-content: space-between;
}

.delete {
```

```
background: red;
color: white;
border: none;
padding: 5px 10px;
border-radius: 5px;
}
```

6. TECHNOLOGIES USED:

- HTML5 – Used to create the webpage structure and user interface.
- CSS3 – Used for styling and layout.
- JavaScript – Used to implement dynamic student management operations.
- DOM Manipulation – Used to dynamically update the student list and count.
- Web Browser – Used to execute and test the application.

7. FOLDER STRUCTURE:

WEEK-3/

```
|
|— index.html
|— script.js
|— style.css
```

Description:

- index.html → Main webpage containing the Student Name Manager interface.
- script.js → Contains JavaScript functions for adding, displaying, counting, and deleting students.
- style.css → Contains CSS styling for the webpage.

8. OUTPUT PROOF:

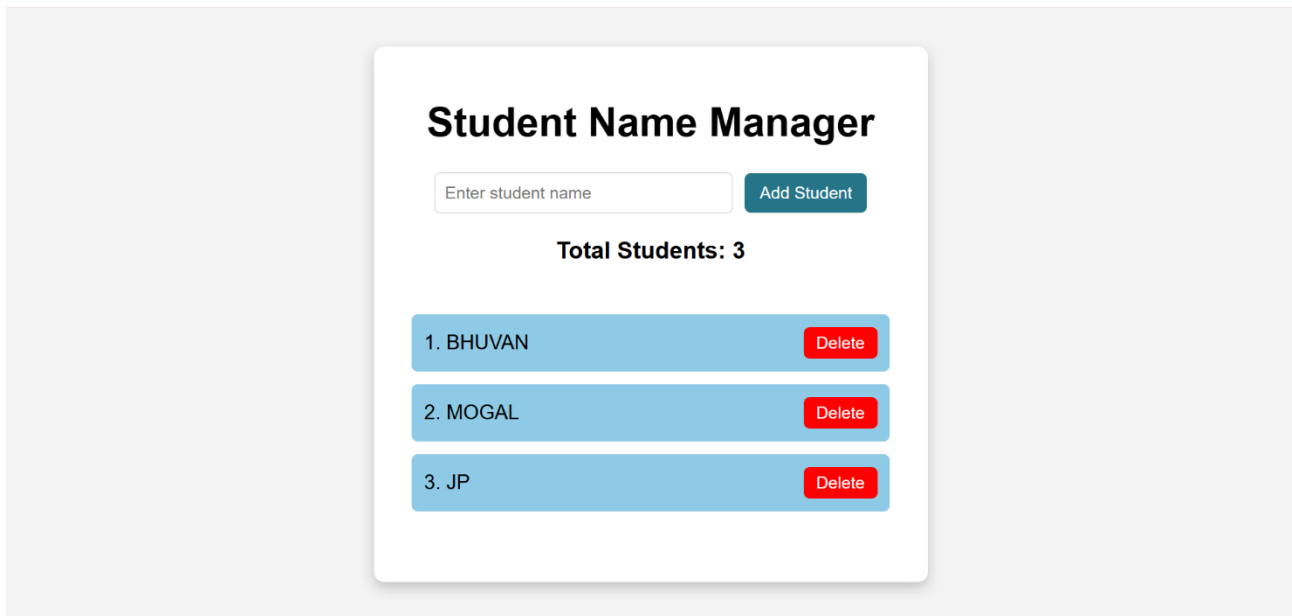
Output: Student Name Manager successfully displayed in the web browser.

Functionalities verified:

- Enter student name
- Add student
- Display student list
- Count total students

- Delete student
- Empty-name validation

Screenshot:



9. LEARNING OUTCOME / RESULT:

Successfully designed and implemented a Student Name Manager using HTML, CSS, and JavaScript. Learned how to use JavaScript arrays, functions, DOM manipulation, event handling, input validation, and dynamic webpage updates.

Result: The Student Name Manager was successfully executed and verified in the browser.